

Individual Final Report: Suicide Ideation Detection Using Bert Model

Yeji Kim

1. Introduction

The objective of the project is to build a text-based classifier that detects whether a social media post indicates suicidal ideation, depressive thoughts, or normal conversation. Many individuals express emotional distress online, yet such signals go unnoticed until it is too late. Thus, our group wants to build a system that detects and flags social media posts suggesting potential emotional distress or risk. We used the suicide and depression dataset from Kaggle (~232k Reddit Posts, and each data included the text and a label as suicide or non-suicide. As we have four members in the team, three people work on the model development (Classical NLP, Deep Learning Model and Transformer) while one person works on demo using streamlit. I choose to work on a transformer model using Bidirectional Encoder Representations from Transformers (BERT).

2. Background and Algorithm Development

BERT (Bidirectional Encoder Representations from Transformer) is a transformer-based language model trained on large English corpora. Unlike RNN, BERT reads text bidirectionally which enables it to understand context more accurately. This makes BERT well-suited for suicide detection where meaning often depends on subtle wording, emotional tone, and context.

The BERT model consists of multiple transformer encoder layers, each containing multi-head self attention mechanisms and feed forward networks. For this project, I used the bert-base-uncased variant, which contains 12 encoder layers, 768 dimensional hidden states, 12 attention heads and approximately 110 million parameters. The model was pretrained on the Toronto Books Corpus and English Wikipedia providing it with broader linguistic knowledge before fine-tuning on our specific task.

2.1. Algorithm and Equations

To adapt BERT for our specific binary classification task, I implemented the following architecture.

2.1.1. Input Representation

Each input sequence begins with a special [CLS] token followed by the tokenized text and ends with a [SEP] token. The input representation combines token embeddings, segment embeddings and positional embeddings

$$h_0 = E_{token} + E_{position} + E_{segment}$$

2.1.2. Contextual Encoding

The input h_0 passes through 12 Transformer blocks. Each block updates the contextual representation of the words using self-attention.

$$h_l = \text{TransformerBlock}(h_{l-1})$$

2.1.3. Classification Head

To repurpose BERT for suicide detection, I extracted the final hidden state of the [CLS] token (h_{CLS}^L). I passed this vector through a custom linear layer and applied a dropout of 0.1 to prevent overfitting.

$$z = Wh_{CLS}^L + b$$

W reduces the dimension from 768 to 2 (Suicide vs Non-Suicide).

2.1.4. Loss Function

During the training, I optimized the model using Cross-Entropy Loss, which penalized the model for confident but wrong prediction.

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^C y_{ik} \log p(y_k | x_i)$$

3. Detailed Description of the Individual Work

3.1. Data Preprocessing

I developed a custom preprocessing script(01_preprocess.py) to standardize the social media text by cleaning text (lowercasing, whitespace removal, removing empty samples, removing duplicate). For the dataset structure, I used a stratified 80/10/10 split (Train/Validation/Test) to maintain consistent class proportion across all phases. After analyzing the distribution of post length, I decide on a maximum sequence length of 320 tokens. It captured the full content of 92% of posts without wasting computational costs.

3.2. Model Training

I built the training infrastructure(02_train.py) using the HuggingFace Trainer API. I loaded bert-base-uncased and fine-tuned the model over 3 epochs with a learning rate of 2e-5 and a batch size of 16. To prevent overfitting, I applied a dropout rate of 0.1 and weight decay of 0.01. I enabled early stopping based on validation loss and saved checkpoints, tokenizer, label mapping and tuned classification threshold. The training pipeline logged metrics throughout and stored the best model in models/bert_base_uncased.

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--model_name", default="bert-base-uncased")
    ap.add_argument("--data_dir", default="data/processed")
    ap.add_argument("--out_dir", default="models/bert_base_uncased")
    ap.add_argument("--max_length", type=int, default=320)
    ap.add_argument("--batch_size", type=int, default=16)
    ap.add_argument("--epochs", type=int, default=3)
    ap.add_argument("--lr", type=float, default=2e-5)
    ap.add_argument("--weight_decay", type=float, default=0.01)
    ap.add_argument("--warmup_ratio", type=float, default=0.06)
    ap.add_argument("--patience", type=int, default=2)
    args = ap.parse_args()
```

3.3. Evaluation and Prediction

To assess the model, I wrote an evaluation script(03_evaluate.py) that computes accuracy, precision, recall, F1 score, ROC-AUC and saved the classification report and confusion matrix. Finally, I implemented the inference script(99_infer.py) which loads these saved results to perform real-time classification on new, unseen text.

3.4. Interpretability and Error Analysis

The interpretability module(04_interpretability.py) represents a significant portion of my individual contribution providing transparency into model behavior.

3.4.1 Attention Aggregation

Since BERT relies on the [CLS] token to classify the entire sequence, I hypothesized that tracking this token's attention would discover which words drove the decision. I implemented a custom aggregation function that calculates the importance scores for every input word. I derived this score by averaging the attention weights from the [CLS] token across all 12 layers and 12 heads.

$$\text{Importance}(t_j) = \frac{1}{L \times H} \sum_{l=1}^L \sum_{h=1}^H A_{CLS,j}^{l,h}$$

```
with torch.no_grad():

    outputs = self.model(**inputs, output_attentions=True)

# attentions: list[num_layers] of (batch, num_heads, seq_len, seq_len)

attentions = outputs.attentions

# average over layers and heads -> (batch, seq_len, seq_len)

avg_attn = torch.stack(attentions).mean(dim=(0, 2))

cls_attention = avg_attn[0, 0, :].cpu().numpy() # CLS → all tokens
```

3.4.2 Keyword Discovery

I implemented a class conditioned keyword extraction that aggregated the attention scores across the entire test set. By separating the suicide and non-suicide, I computed the cumulative attention weight for every unique token in the vocabulary. This allowed me to calculate a differential score identifying words that trigger a suicide prediction versus those that signal safety.

3.4.3. Error Analysis

To diagnose the model's error, I isolated all false positives and false negatives into separate csv files. I also implemented a quantitative check to verify that these errors were not the result of text length. I calculated character and word counts for every prediction and visualized the distribution using boxplots.

4. Results

The fine-tuned BERT model gave impressive results on the held-out test set of 23,123 posts. Table 1 summarizes the key performance metrics.

Table1.

	precision	recall	f1-score	support
0	0.9827	0.9776	0.9801	11595
1	0.9777	0.9828	0.9802	11604
accuracy			0.9802	23199
macro avg	0.9802	0.9802	0.9802	23199
weighted avg	0.9802	0.9802	0.9802	23199

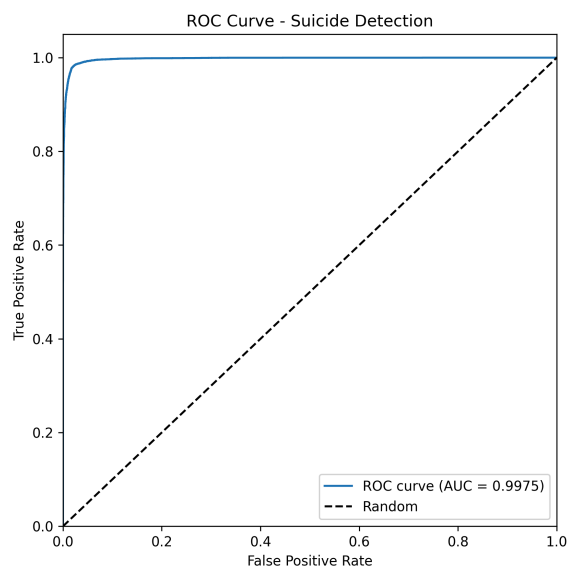
ROC_AUC: 0.9975

The results indicated that the model achieves 98% accuracy while maintaining balanced precision and recall. The AUC scores (0.9975) demonstrate excellent discrimination capability suggesting that the model has learned meaningful patterns that generalize well to unseen data. Also, the balanced precision and recall show that the model avoids bias toward either error type.

4.1 ROC Curve

The receiver operating characteristic (ROC) curve plots the true positive rate against the false positive rate across all possible classification thresholds. The curve hugs the top-left corner of the plot space. This plot confirmed that suicidal and non-suicidal posts from well separated distributions in the model's learned representation space.

Figure 1. ROC Curve

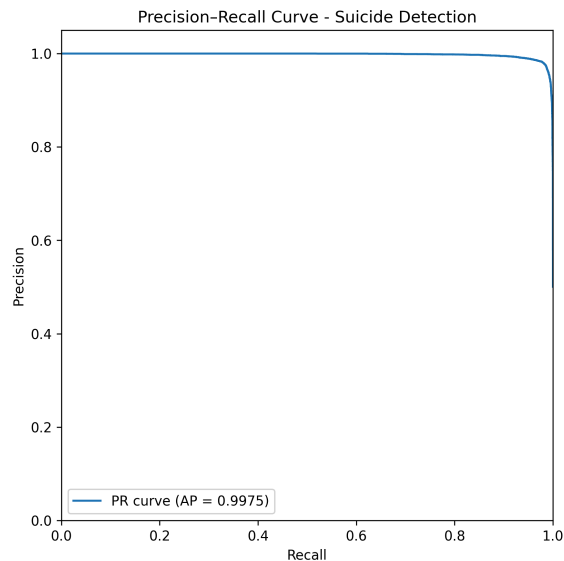


4.2 The Precision-Recall Curve

The precision recall curve provides a detailed view of model performance on the positive class. As shown in the figure, the curve maintains precision above 0.95 across the full range of recall values indicating that it rarely misclassifies non-suicidal posts as suicidal. The Average

Precision(AP) scores of 0.9975 illustrates that the classifier ranks suicidal posts ahead of non-suicidal posts with remarkable consistency. The model is able to retrieve the majority of suicidal posts without introducing a substantial number of false alarms. Thus, the PR curve confirms that the classifier is both highly sensitive and precise in distinguishing it from non-suicidal content.

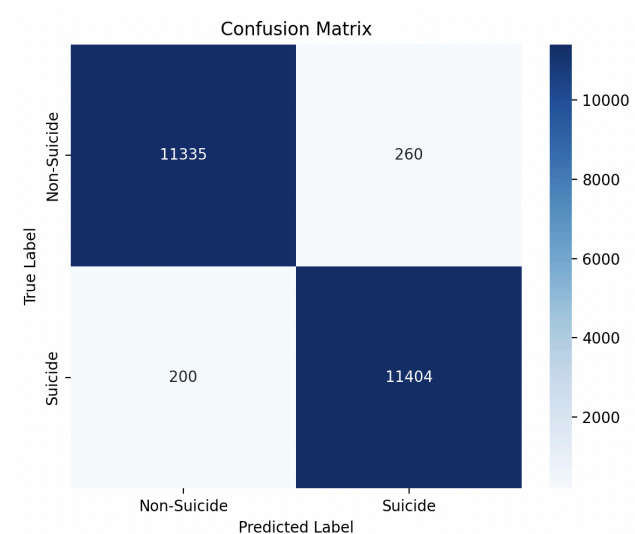
Figure 2. PR Curve



4.3. Confusion Matrix

The confusion matrix reveals detailed information of model performance across the two classes. The model correctly identified 11,335 non-suicidal posts and 11,404 suicidal posts indicating strong reliability across both categories. False positive (260 cases) represent posts that the model flagged as suicidal despite being non suicidal. False negative (200 cases) corresponds to suicidal posts that are missed. Since false positive and false negative rates are low and balanced, this result shows that the model is robust across all classes.

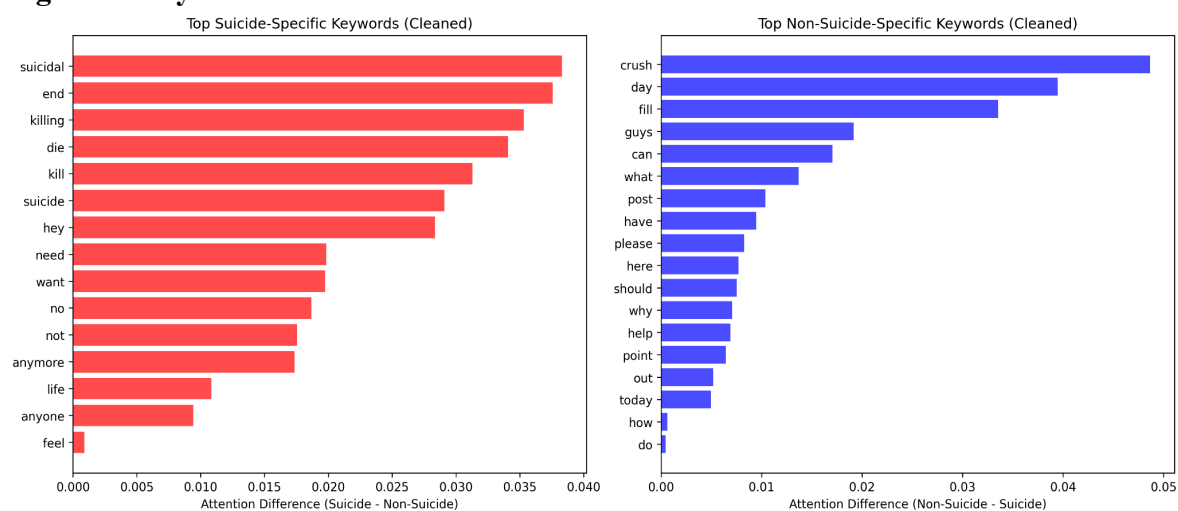
Figure 3. Confusion Matrix



4.4. Interpretability

Using the attention aggregation method described in 3.4.1, I successfully identified the linguistic features driving the model's predictions. As illustrated in Figure 4, it presents the top suicidal and non-suicidal keywords after stopword and punctuation filtering. The highest scoring token such as suicidal, end, killing, die, kill, suicide correspond to explicit indicators of suicidal ideation. These words directly describe self-harm or death related expressions. The clear semantic separation between the two keyword groups illustrates that the model's decisions are grounded in meaningful linguistic distinctions.

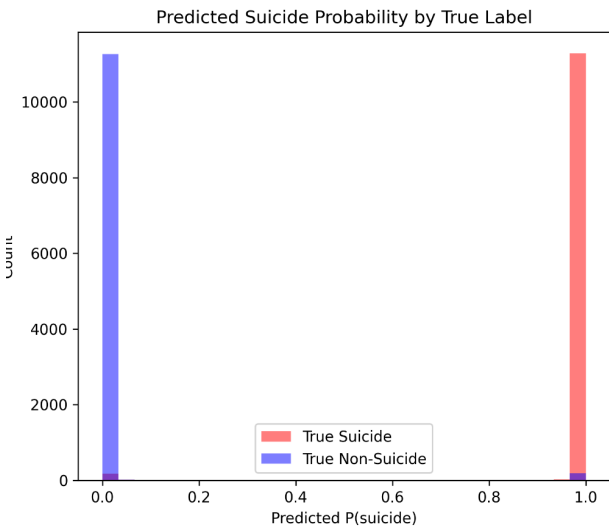
Figure 4. Keyword



4.5 Error Analysis

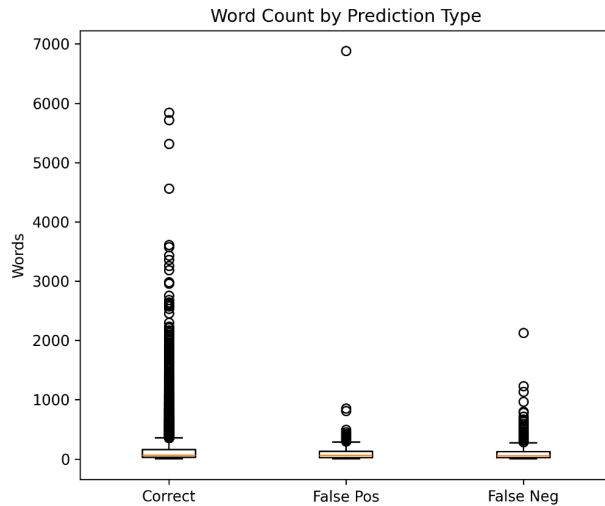
I conducted a comprehensive analysis of the model’s failures to understand its limitations. I conducted quantitative and qualitative error analysis. Two figures below show how and why misclassification occurs. The histogram in Figure 5 shows the distribution of predicted suicide probabilities for true suicidal and true non suicidal posts. The separation pattern is consistent with the model’s high ROC-AUC score showing the classifier maintains strong discriminative capability across the full probability spectrum.

Figure 5. Predicted Suicide Probability Distribution



The figure 6 compares the word count distribution among correct predictions, false positives and false negatives. While the model performs generally well across different lengths, the error analysis reveals a specific vulnerability in short texts. Short inputs containing informal language or slang lack the contextual redundancy found in longer posts leading to false negatives. In longer posts, the model can recover from slang terms by relying on surrounding context words. Additionally, errors arise from semantic and emotional complexity such as posts using sarcasm, metaphors or indirect expressions of distress without explicit keywords.

Figure 6. Word Count by Prediction Type



5. Conclusions

The project demonstrates that modern NLP techniques can achieve high performance on suicide ideation detection while maintaining interpretability. The classifier achieved high accuracy, strong recall and excellent AUC scores. Through this project, I gained experience with transformer fine-tuning, dataset preparation, interpretability analysis and performance evaluation. The result shows that BERT is highly effective for suicide ideation detection. However, there are subtle or indirect expressions remaining challenging. For future work, I would use larger or domain-specific pretrained models such as mental-health-specific BERT variants or RoBERTa-large. Also, I would use contrastive learning or emotion aware representations to improve performance on subtle expressions and enhance interpretability.

6. Code Originality Statement

The full project codebase consists of 1,444 lines of Python covering data fetching, preprocessing, training, evaluation, interpretability, visualization and inference. Approximately 1,194 lines were adapted from online sources documentation templates or AI assisted drafts. I substantially modified about 200 of those lines. In addition, I wrote 250 lines of entirely original code for dataset specific preprocessing, evaluation logic, threshold and interpretability.

$$\begin{aligned}
 \text{Internet Percentage} &= \frac{\text{Lines Found} - \text{Lines Modified}}{\text{Lines Found} + \text{Lines Added}} \times 100 \\
 &= \frac{1194 - 200}{1194 + 250} \times 100 \\
 &= \frac{994}{1444} \times 100 \\
 &\approx 68.8\%
 \end{aligned}$$

$$\begin{aligned}\text{Original Contribution} &= 100\% - 68.8\% \\ &= \mathbf{31.2\%}\end{aligned}$$

7. Reference

- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv preprint arXiv:1810.04805.
- Wolf, T., et al. (2019). *HuggingFace's Transformers: State-of-the-art Natural Language Processing*. arXiv preprint arXiv:1910.03771.
- Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.
- Komati, N., Vishal, D., Sphoorthi, L., & Fathimabi, S. (2021). *Suicide Ideation Detection in Social Media Forums*. 2021 2nd International Conference on Smart Electronics and Communication (ICOSEC), 1741–1747.
<https://doi.org/10.1109/ICOSEC51865.2021.9591887>